

Level 1 - Understanding

Q1. File-system problems

The two problems are data redundancy and data inconsistency. A database keeps the data in one central place, so the same address does not need to be stored many times. When the address is changed, everyone sees the updated value.

Q2. Three things a DBMS does that a file system does not

- Data consistency - avoids different departments having different customer details.
- Security - controls who can view or change sensitive banking data.
- Transaction management - makes sure money transfers are completed properly.

Q3. Relational terms

Formal term	Everyday term	ICICI example
Relation	Table	Customer table
Tuple	Row	One customer's details
Attribute	Column	Customer name
Degree	Number of columns	Number of columns in customer
Cardinality	Number of rows	Number of customers

Q4. Identify the type of key

- customer.customer_id - Primary key
- account.account_number - Primary key
- account.customer_id - Foreign key
- account.balance - None
- employee.manager_id - Foreign key
- branch.ifsc_code - Candidate key

Q5. OLTP or OLAP

- 1. Balance checking - OLTP: quick real-time transaction.
- 2. NPL by branch for quarter - OLAP: analysis of large data.
- 3. ATM withdrawal - OLTP: real-time transaction.
- 4. Transactions above Rs 2,00,000 - OLAP: reporting and analysis.
- 5. Opening savings account - OLTP: creates a new record/transaction.
- 6. Top 100 customers by balance - OLAP: data analysis.

Q6. Why not run the quarterly report directly on the core Oracle database?

- Large reports can slow down the main banking system.
- Reports use CPU, memory and database resources needed for customer transactions.
- It can create performance and security problems.
- Offer instead - a reporting database, data warehouse or read replica.

Q7. ACID property matching

- Atomicity -> d. Money is debited but never credited.
- Consistency -> c. An account exists for a customer who does not.
- Isolation -> b. Two ATMs both pay out against the same balance.
- Durability -> a. A committed transfer disappears after a power cut.

Q8. Why is the write-ahead log written first?

The log is written to disk first so the database can recover the transaction after a crash or power failure. If the data page were written first, the database could lose track of which changes were completed and committed.

Q9. SELECT * and data independence

It breaks logical data independence because the application depends on the table structure. If a new 'pan_number' column is added, the result set changes and old application code may not handle it properly.

Q10. Where would you use Oracle, PostgreSQL, MySQL and Redis?

- Oracle - core banking systems where strong reliability and transactions are important.
- PostgreSQL - applications that need a powerful relational database.
- MySQL - web applications and general-purpose database workloads.
- Redis - fast caching and frequently accessed temporary data.

Level 2 - Applying SQL

Q11. Database name, current user and server port

```
SELECT current_database(), current_user, inet_server_port();
```

The output shows the database name, current user and PostgreSQL server port.

Q12. List every column of the loan table

```
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'loan'
ORDER BY ordinal_position;
```

Q13. Number of rows in all nine tables

```
SELECT 'account' AS table_name, COUNT(*) AS row_count FROM account
UNION ALL SELECT 'account_txn', COUNT(*) FROM account_txn
UNION ALL SELECT 'beneficiary', COUNT(*) FROM beneficiary
UNION ALL SELECT 'branch', COUNT(*) FROM branch
UNION ALL SELECT 'card', COUNT(*) FROM card
UNION ALL SELECT 'customer', COUNT(*) FROM customer
UNION ALL SELECT 'employee', COUNT(*) FROM employee
UNION ALL SELECT 'loan', COUNT(*) FROM loan
UNION ALL SELECT 'loan_payment', COUNT(*) FROM loan_payment
ORDER BY table_name;
```

Q14. List every foreign key

```
SELECT
tc.table_name AS child_table,
kcu.column_name AS child_column,
ccu.table_name AS parent_table
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu
ON tc.constraint_name = kcu.constraint_name
JOIN information_schema.constraint_column_usage ccu
ON tc.constraint_name = ccu.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY'
ORDER BY tc.table_name;
```

Q15. Three largest tables on disk

```
SELECT relname AS table_name,
pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 3;
```

Q16. Insert customer 1032 and account 9047

```
INSERT INTO customer
(customer_id, name, date_of_birth, email, phone, city, kyc_status, risk_rating, customer_since, home_branch)
VALUES
(1032, 'Latha Krishnan', '1985-09-23', 'latha.krishnan@example.com',
'9845099887', 'Chennai', 'VERIFIED', 'MEDIUM', '2026-03-01', 5);
```

```
INSERT INTO account
(account_id, account_number, customer_id, account_type, balance, opened_date, interest_rate)
VALUES
(9047, 'ICIC0000009047', 1032, 'CURRENT', 500000.00, '2026-03-01', 0.00);
```

```
SELECT * FROM customer WHERE customer_id = 1032;
```

```
SELECT * FROM account WHERE account_id = 9047;
```

Q17. Deposit Rs 75,000 and withdraw Rs 20,000

```
UPDATE account
SET balance = balance + 75000
WHERE account_id = 9047;
```

```
UPDATE account
SET balance = balance - 20000
WHERE account_id = 9047;
```

```
SELECT account_id, balance
FROM account
WHERE account_id = 9047;
```

Final balance: Rs 5,55,000.

Q18. Delete customer 1032 in the correct order

```
DELETE FROM customer
WHERE customer_id = 1032;
```

This should fail with a foreign key constraint error because account `9047` still refers to customer `1032`.

```
DELETE FROM account
WHERE account_id = 9047;
```

```
DELETE FROM customer
WHERE customer_id = 1032;
```

```
SELECT COUNT(*) FROM customer;
SELECT COUNT(*) FROM account;
```

Expected counts: Customer = 30, Account = 45.

Q19. Statements the database must reject

The following are example statements. Your database may show a specific constraint name in the error message.

```
INSERT INTO account
(account_id, account_number, customer_id, account_type, balance, opened_date, interest_rate)
VALUES
```

```
(9999, 'TEST9999', 1, 'FIXED', 1000, CURRENT_DATE, 0);
```

Reason: `FIXED` is not an allowed account type.

```
INSERT INTO account_txn  
(amount)  
VALUES (-5000);
```

Reason: the amount cannot be negative.

```
INSERT INTO customer  
(customer_id, name, kyc_status)  
VALUES (9999, 'Test Customer', 'IN_PROGRESS');
```

Q20. Uncommitted UPDATE in two sessions

Session A:

```
BEGIN;  
UPDATE account  
SET balance = balance + 1000  
WHERE account_id = 9004;  
SELECT balance  
FROM account  
WHERE account_id = 9004;
```

Do not commit the transaction in Session A.

Session B:

```
SELECT balance  
FROM account  
WHERE account_id = 9004;
```

Session B still sees the old balance because Session A has not committed its update.

Q21. Transfer Rs 12,000 and roll it back

```
SELECT account_id, balance  
FROM account  
WHERE account_id IN (9004, 9010);
```

```
BEGIN;
```

```
UPDATE account  
SET balance = balance - 12000  
WHERE account_id = 9004;
```

```
UPDATE account  
SET balance = balance + 12000  
WHERE account_id = 9010;
```

```
ROLLBACK;
```

```
SELECT account_id, balance  
FROM account  
WHERE account_id IN (9004, 9010);
```

The balances after the rollback should be the same as before the transaction.

Q22. Show important PostgreSQL settings

```
SHOW shared_buffers;  
SHOW work_mem;  
SHOW wal_level;  
SHOW fsync;
```

- shared_buffers - memory used for database data.
- work_mem - memory used for sorting, joins and similar operations.
- wal_level - controls how much information is written to WAL.
- fsync - controls safe writing of data to disk.

Level 3 - Analysis and Design Decisions

Q23. MyISAM for a high-volume UPI system

I would not recommend MyISAM for UPI transactions because it does not provide the transaction support needed for banking transfers. A better alternative would be InnoDB, which supports transactions, row-level locking and crash recovery.

Q24. 40 million transaction rows and two different workloads

One database design may not handle both requirements efficiently because branch users need fast access to individual transactions, while the risk team needs large-scale analysis. I would use the operational database for branch transactions and a separate analytical database or data warehouse for reporting.

Q25. Why an application log does not prove a transfer succeeded

An application log only shows what the application reported; it does not prove that the database transaction was committed. The stronger evidence is the database transaction status and account/transaction records showing that the debit and credit were actually committed.

Q26. Setting fsync = off

I would not set fsync = off on the core banking database because a crash could cause committed data to be lost or corrupted. It could be considered only for a temporary test/development system where data loss is acceptable and real banking transactions are not handled.

Q27. Why keep check constraints in the database?

- The database protects the rule even if a different application tries to insert wrong values.
- The same rule is applied to every application using the database.
- Application layer is better when - the rule depends on user interaction or complex business logic.

Q28. Oracle vs PostgreSQL differences during a merger

- SQL syntax and functions can be different.
- Data types are not exactly the same.
- Transaction and locking behavior can differ.
- Backup, replication and administration tools are different.

Q29. Reply to branch operations head

Moving from spreadsheets to a relational database will keep customer and account information in one controlled system instead of many separate files. This reduces duplicate information and avoids different departments having different versions of the same data. It also allows the bank to control who can view or change information. The database can make sure that account and customer records are properly connected.

It can also prevent invalid values from being entered. However, moving to a database will not automatically make incorrect data correct. It will also not prevent mistakes made by employees or poorly designed applications. The bank must define clear data rules and access permissions. Employees also need proper training on the new

system. Regular backups and checks should be performed. Any existing incorrect or duplicate data should be cleaned before migration.